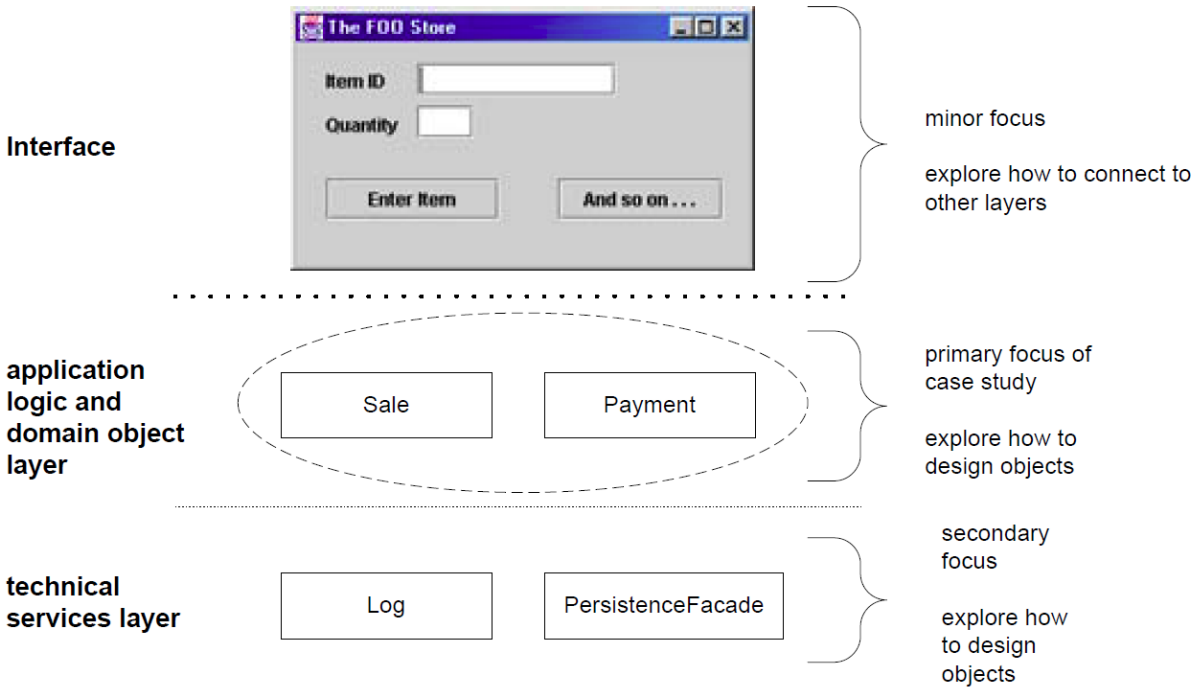
Chapter 3 Case Studies

이찬근

소프트웨어학부
중앙대학교

What is and isn't Covered in the Case Studies?

- Generally, applications include
  - UI elements,
  - Core application logic,
  - Database access and
  - Collaboration with external software or hardware components.



What is and isn't Covered in the Case Studies?

- OO technology can be applied at all levels.

- But, this book focuses on the core application logic layer, with some secondary discussion of the other layers.
  - Other layers are usually very technology/platform dependent.
  - In contrast, the OO design of the core logic layer is similar across technologies.
  - The essential OO design skills learned in the context of the application logic layer are applicable to all other layers or components.
  - The design approach/patterns for the other layers tends to change quickly as new frameworks or technologies emerge.
    - e.g., in-house access layer for database -> open-source solution.

Case One: The NextGen POS System

The first case study is the NextGen point-of-sale (POS) system. In this apparently straightforward problem domain, we shall see that there are interesting requirement and design problems to solve. In addition, it's a real problemgroups really do develop POS systems with object technologies.

A POS system is a computerized application used (in part) to record sales and handle payments; it is typically used in a retail store. It includes hardware components such as a computer and bar code scanner, and software to run the system. It interfaces to various service applications, such as a third-party tax calculator and inventory control. These systems must be relatively fault-tolerant; that is, even if remote services are temporarily unavailable (such as the inventory system), they must still be capable of capturing sales and handling at least cash payments (so that the business is not crippled).

A POS system increasingly must support multiple and varied client-side terminals and interfaces. These include a thin-client Web browser terminal, a regular personal computer with something like a Java Swing graphical user interface, touch screen input, wireless PDAs, and so forth.

Furthermore, we are creating a commercial POS system that we will sell to different clients with disparate needs in terms of business rule processing. Each client will desire a unique set of logic to execute at certain predictable points in scenarios of using the system, such as when a new sale is initiated or when a new line item is added. Therefore, we will need a mechanism to provide this flexibility and customization.

Using an iterative development strategy, we are going to proceed through requirements, object-oriented analysis, design, and implementation.

Case Two: The Monopoly Game System

To show that the same practices of OOA/D can apply to *very* different problems, I've chosen a software version of the game of Monopoly® as another case study. Although the domain and requirements are not at all like a business system such as the NextGen POS, we will see that domain modeling, object design with patterns, and applying the UML are still relevant and useful. As with a POS, software versions of Monopoly are truly developed and sold, with both rich client and Web UIs.

I won't repeat the rules for Monopoly; it seems almost every person, in every country, has played this game as a child or teenager. If you have questions, the rules are available online at many websites.

The software version of the game will run as a simulation. One person will start the game and indicate the number of simulated players, and then watch while the game runs to completion, presenting a trace of the activity during the simulated player turns.